Temporal Databases

Time

“It's present everywhere, but occupies no space.
We can measure it, but we can't see it, touch it,
get rid of it, or put it in a container.
Everyone knows what it is and uses it every day,
but no one has been able to define it.
We can spend it, save it, waste it, or kill it,
but we can't destroy it or even change it,
and there's never any more or less of it.”

~~Je~~spersen and Fitz-Randolph, From Sundials to Atomic Clocks

Time Structure

- Linear: total order on instants
- Hypothetical (possible futures): tree rooted on now
- Directed Acyclic Graph (DAG): possible futures may merge
- Periodic/cyclic time: weeks, months…
- Assume a linear time structure
- Boundedness
  - Unbounded
  - Time origin exists (bounded from the left)
  - Bounded time (bounds on two ends)
- Nature of bound
  - Unspecified
  - Specified

Time Density

- Discrete
  - Time line is shaped by integers
  - Time line is composed of a sequence of non-decomposable time periods, of some fixed minimal duration, termed chronons
  - Between each pair of chronons is a finite number of other chronons
- Dense
  - Time line is shaped by rational numbers
  - Infinite number of instants between each pair of chronons
- Continuous
  - Time line is shaped by real numbers
  - Infinite number of instants between each pair of chronons
- Distance may optionally be defined

What is temporal DB?

- Temporal databases include all DB applications that require some aspect of time when organizing their information.
- Temporal DB applications have been developed since the early days of database usage. However, in creating these applications, it was mainly left to the application developers to discover, design, program, and implement the temporal concepts.

Why now?

- Decreasing cost of storage
- Data warehouse technology adoption in the majority of the domains
- The need of maintaining and processing historical data (legal, security, etc...)

Why need temporal data?

Two questions should answer:

- Does your organization need to know the situation as it was known at a particular date (e.g. aviation - the fuel price for a certain date)?

- Does your business use information that was effective in the past or will become effective in the future (e.g. the new address of the customer)?

Temporal DBs — Motivation

- Conventional databases represent the state of an enterprise at a single moment of time (snapshot)
- Many applications need information about the past (time-varying data)
  - Banking (account transactions to compute balance)
  - Medical (patient history)
  - Financial (payroll)
- Temporal DBs: a system that manages time varying data

Why time varying data?

Examples of application domains dealing with time varying data:

- Financial Apps (e.g. history of currency exchange data)
- Insurance Apps (e.g. when were the policies in effect)
- Reservation Systems (e.g. when is which room in a hotel booked)
- Medical Information Management Systems (e.g. patient records)
- Decision Support Systems (e.g. planning future contingencies)
- HR applications (e.g. Date tracked positions in hierarchies)

Comparison

Conventional DBs:

— Evolve through transactions from one state to the next

— Changes are viewed as modifications to the state

— No information about the past

— Snapshot of the enterprise

Temporal DBs:

— Maintain historical information

— Changes are viewed as additions to the information stored in the database

— Incorporate notion of time in the system

— Efficient access to past states

Temporal Databases

- Temporal Data Models: extension of relational model by adding temporal attributes to each relation
- Temporal Query Languages: TQUEL, SQL3
- Temporal Indexing Methods and Query Processing

Temporal Databases

- Temporal data types
- Kinds of time
- Temporal statements
- There are three fundamental temporal data types:
  - Instant
  - Interval
  - Period

Instant

- Instant: something happened at an instant of time: eg, now, 30th of September 2025, 19:30, as we speak at the course
- An instant is an anchored location on the timeline. An SQL-92 provides: DATE (a particular day, with a year in the range ad 1–9999), TIME (a particular second within a range of 24 hours), and TIMESTAMP (a particular fraction of a second, defaulting to microsecond, of a particular day).

(Time) Interval

- A length of time (e.g., three months)
- An interval is relative; an instant is absolute. An interval can be added to an instant, yielding another instant.
- A time interval is a directed duration of time, an amount of time with a known length, but without exact starting or ending instants.
- The distance between two instants is an interval. Unlike instants, intervals have direction. An interval can be positive or negative, denoting a shift to the future or to the past.

Interval

- The SQL-92 interval type is complex. SQL differentiates year-month intervals and day-time intervals. The first,an integral number of years or months; the latter, an integral number of days, hours, minutes, seconds, or fractions of a second. This distinction is due to varying month lengths in the Gregorian calendar.
- The individual units (months, hours, seconds) are termed granules, so an interval value is a (signed) integer number of granules.
- Intervals are combinations of the fields year, month, day, hour, minute, and second, though not all combinations are allowed.

Period

- An instant has no duration. Yet facts in the database are true over a duration of time. To express when a fact holds in the enterprise, a period is associated with that fact.
- Period: an anchored duration of time (e.g., the spring semester, 29th of September 2025 through 27th of February, 2026)
- SQL-92 includes support for instants and intervals. Most DBMS products, though, only support instants, with intervals being simulated with integers or floating-point numerics. Periods are always left to the application developer to simulate using supplied data types.
- As SQL-92 does not provide a period data type, there are no period literals in that language. Periods must instead be specified by their constituent datetime and interval literals.

Time Predicates

- For such a diverse set of types (DATE, TIME, TIMESTAMP, TIME WITH TIME ZONE, TIMESTAMP WITH TIME ZONE, and two variants of INTERVAL: year-month and day-time), SQL-92 supports only four classes of temporal predicates: equality, less than, is null, and overlaps.
- There are several variants of the equality predicate; these variants apply to all the data types. When applied to two expressions, '**=**' determines whether the values of these expressions are identical. When applied to a value and a set of values (of the same type), **=ANY** determines if the left-hand value is identical to at least one of the values in the right-hand set. **=SOME** and IN are non orthogonal equivalents. **MATCH** also relies on equality testing.

Taxonomy of time

- *User defined time*: an uninterpreted time value
- *Transaction time* databases:

  — Transaction time is the time when a fact is stored in the database
- *Valid time* databases:

  - Valid time is the time that a fact becomes effective in reality
- *Bi-temporal* databases:

  — Support both notions of time

Temporal databases

- We can use these two dimensions to distinguish between different forms of temporal database
  - A rollback database stores data with respect to transaction time
  - A historical database stores data with respect to valid time
  - A bi-temporal database stores data with respect to both valid time and transaction time.

Temporal Database Features

Temporal databases support managing and accessing temporal data by providing one or more of the following features:

- A time period datatype, including the ability to represent time periods with no end (infinity or forever)
- The ability to define valid and transaction time period attributes and bitemporal relations
- System-maintained transaction time

Temporal Database Features

- Temporal primary keys, including non-overlapping period constraints
- Temporal constraints, including non-overlapping uniqueness and referential integrity
- Update and deletion of temporal records with automatic splitting and coalescing of time periods
- Temporal queries at current time, time points in the past or future, or over durations
- Predicates for querying time periods

Example

- Sales example: data about sales are stored at the end of the day
- Transaction time is different than valid time
- Valid time can refer to the future also!
  - Credit card: O1/2026-O1/2030

Transaction Time DBs

- Time evolves in discrete manner, usually is associated with the transaction number (or SCN):

  T1 -> T2 -> T3 -> … -> Tn

- A record R is extended with an interval [t.start, t.end). When we insert an object at t1 the temporal attributes are updated -> [t1, now)
- Updates can be made only to the current state!
  - Past cannot be changed
  - “Rollback” characteristics

Transaction Time DBs

- Transaction time records the time period during which a database entry is accepted as correct.
- This enables queries that show the state of the database at a given time.
- Transaction time periods can only occur in the past or up to the current time.
- In a transaction time table, records are never deleted. Only new records can be inserted, and existing ones updated by setting their transaction end time to show that they are no longer current.

Transaction Time DBs

- Deletion is logical (never physical deletions)
  — When an object is deleted at t2, its temporal attribute changes from [t1, now) -> [t1, t2)
  — Object is “alive” from insertion to deletion time, ex. t1 to t2. If “now” then the object is still alive

eid	salary	start	end
10	20K	9/93	10/94
20	50K	4/94	*
33	30K	5/94	6/95
10	50K	1/95	*



Transaction Time DBs

- Requirements for index methods:
  - — Store past logical states
  - — Support addition/deletion/modification changes on the objects of the current state
  - — Efficiently access and query any database state

Transaction Time DBs

- Queries:

  - Timestamp (timeslice) queries: ex. “Give me all employees at 09/2025”

  - Range-timeslice: “Find all employees with id between 100 and 200 that worked in the company on 09/2025”

  - Interval (period) queries: “Find all employees with id in [100,200] from 01/2023 to 09/2025”

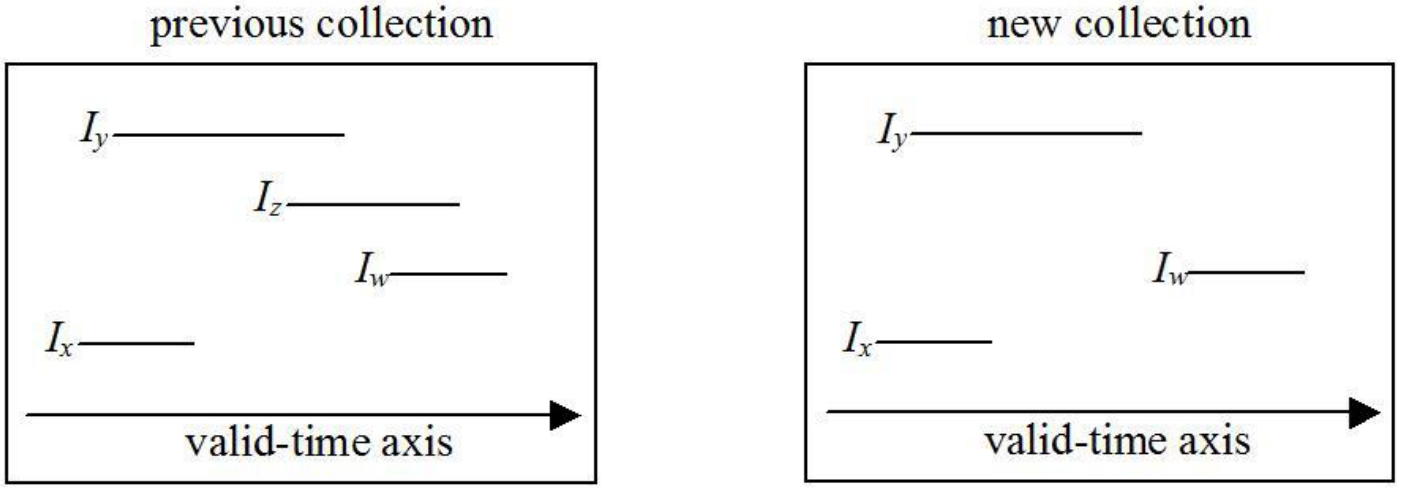
Valid Time DBS

- Valid time is the time for which a fact is true in the real world.
- A valid time period may be in the past, contain the current time, or occur in the future.
- Time evolves continuously
- Each object is a line segment representing its time span (eg. Credit card valid time)
- Physical deletion is possible
- Support full operations on interval data:
  - Deletion at any time
  - Insertion at any time
  - Value change (modification) at any time (no ordering)

Valid Time

- Event table
  - single chronon identifier
- State table
  - interval timestamps

Valid Time DBS



The reality "best know now !"

Valid Time DBS

- The time interval is closed at its lower bound and open at its upper bound
- Requirements for an Index method:

— Store the latest collection of interval-objects

— Support add/delete/modify changes to this collection

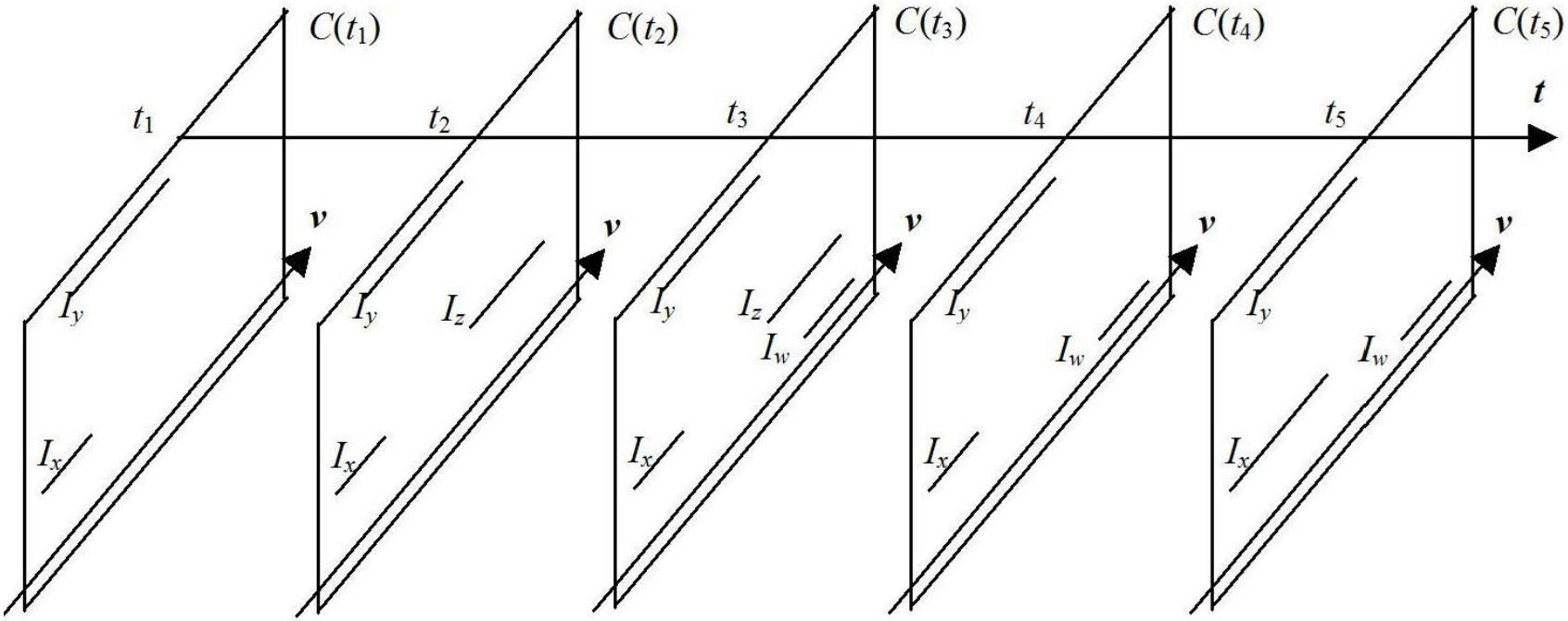
— Efficiently query the intervals in the collection

- Timestamp query
- Interval (period) query

Bi-temporal DBs

- A bi-temporal relation contains both valid and transaction time. This provides both historical and rollback information. Historical information (e.g.: "Where did John live in 1992?") is provided by the valid time. Rollback (e.g.: "In 1992, where did the database believe John lived?") is provided by the transaction time.
- It is a transaction-time database as each record is an interval (plus the other attributes of the record)
- At each timestamp, it is a valid time database

Bi-temporal DBs



Examples

- Model a car hire problem
  - as event table
  - as state table
- Ideas how to store transaction time for transactions that affect several tables?

SQL 2011 Standard

- There is no commonly accepted definition of temporal data. Let us consider temporal data to be any data with one or more associated time periods during which that data is considered to be in effect or valid.
- A (SQL) standard has to deal amongst others with:
  - data type and table definition standardization,
  - table definition modification,
  - (referential) integrity enforcement
  - data manipulation

SQL 2011 Standard

- A time period is a mathematical interval on the time line, bounded by a start time and an end time.
- It was introduced a period data type, defined as an ordered pair of two datetime values, for the purpose of associating time periods with the rows of a table. **SQL-2011 has not adopted this solution.**
- Adding a new data type to the SQL standard (or to an SQL product) is a costly operation because of the new data type has to be supported also in all the software that form the ecosystem surrounding SQL.

SQL 2011 Standard

- Instead of adding a period type SQL 2011 adopts period definitions as **table meta data**.
- The model adopted is a closed-open period model.
- A period definition as table component identifies a pair of columns that store the period start and end time.
- CREATE TABLE and ALTER TABLE statements are enhanced with **syntax for creating or modifying period definitions**.
- The period start and end columns are conventional columns, with separate names.
- The period name occupies the same name space as column names, a period can't have the same name as a column.

SQL 2011 Standard

- *transaction time support* provided by system versioned tables. The name of the system time period is specified by the standard as *SYSTEM_TIME*.

- *valid time support* provided by tables storing application time period. The name of an application time period can be any *user-defined* name.

- can be defined at most one application time period and at most one system time period per table.

- allows existing databases that store periods using a pair of date time columns to take advantage of the SQL-2011 extensions more easily.

- as most DBMSs do not support a period type, the period information using a pair of columns of datetime data type is the solution.

SQL 2011 Standard

- Application (valid) time period: applications that are interested in storing time periods during which the data is believed to be valid in the real world.
- Example: insurance application, keep track of the specific policy details of a given customer that are in effect at any given point in time.
- User be free to assign any time values, either in the past, current or in the future, for the start and end times.
- User be permitted to update the validity periods of the rows as errors are discovered or new information is made available.
- Any table that contains a period definition with a user-defined name is an application-time period table.

SQL 2011 Standard

```
CREATE TABLE Emp(
ENo INTEGER,
EStart DATE,
EEnd DATE,
EDept INTEGER,
PERIOD FOR EPeriod (EStart, EEnd)
)
```

Users can pick any name for the name of the period as well as for the names of columns that act as the start and end of the period.

The data types of the period start and end columns must be either **DATE** or a **timestamp** type, and data types of both columns must be the same.

SQL 2011 Standard

```
INSERT INTO EmpVALUES (22217, DATE ‘2010-01-01’,DATE '2011-11-12', 3)
```

- The conventional UPDATE statement can be used to modify the rows of application-time period tables.
- The conventional DELETE statement can be used to delete rows of application-time period tables.

New feature of SQL-2011 is the ability to specify changes that are effective within a specified period.

Provided by a syntactic extension to both UPDATE and DELETE statements lets users specify the period of interest.

```
UPDATE Emp FOR PORTION OF EPeriod FROM DATE '2011-02-03' TO DATE '2011-09-10'SET EDept = 4 WHERE ENo = 22217
```

SQL 2011 Standard

- To execute this statement, the DBMS locates all rows whose application-time period overlaps the period P from Feb. 3, 2011 to Sept. 10, 2011; (periods follow closed-open semantics in SQL-2011) so P includes Feb. 3, 2011 but excludes Sept. 10, 2011. Any overlapping row whose application-time period is contained in P is simply updated.
- If an overlapping row whose application-time period has a portion either strictly before or strictly after P, then that row gets split into two or three contiguous rows depending on the extent of overlap, and of these, the row whose application-time period is contained in P is updated.

SQL 2011 Standard

- The DELETE statement is similarly enhanced with **FOR PORTION OF** syntax to facilitate deletes that are only effective within a specified period.

```
DELETE Emp FOR PORTION OF EPeriod FROM DATE
'2011-02-03' TO DATE '2011-09-10' WHERE ENo
= 22217
```

- Similar to the UPDATE example, any row whose application-time period is contained in P from Feb. 3,2011 to Sept. 10, 2011 is simply deleted. If an overlapping row whose application-time period has a portion either strictly before or strictly after P, then that row gets split into two or three contiguous rows, and of these, the row whose application-time period is contained in P is deleted.

SQL 2011 Standard

It must be possible to forbid overlapping application-time periods:

```
ALTER TABLE EmpADD PRIMARY KEY (ENo, EPeriod
WITHOUT OVERLAPS)
```

Consider the following definition:

```
CREATE TABLE Dept(
DNo INTEGER,
DStart DATE,
DEnd DATE,
DName VARCHAR(30),
PERIOD FOR DPeriod (DStart, DEnd),
PRIMARY KEY (DNo, DPeriod WITHOUT OVERLAPS))
```

SQL 2011 Standard

- Assume also that we want to make sure that at every point in time, every value in EDept column corresponds to some value of DNo column in Dept table, i.e., every employee at every point in time during her employment belongs to a department that actually exists at that point in time. How should this work?
- It must be possible to forbid a row in a child table whose application-time period is not contained in the application-time period of a matching row in the parent table, which can be specified with this syntax:

```
ALTER TABLE EmpADD FOREIGN KEY (Edept,
PERIOD EPeriod)  REFERENCES Dept (DNo,
PERIOD DPeriod)
```

SQL 2011 Standard

With this referential constraint definition, the sample data is prohibited as a constraint violation. More generally, for a given child row, *it is not necessary that there exists exactly one matching row in the parent table* whose application-time period contains the application-time period of the child row. As long as the application-time period of a row in the child table is contained in the *union of application-time periods* of two or more contiguous matching rows in the parent table, the referential constraint is considered satisfied.

SQL 2011 Standard

In SQL-2011, application-time period tables can be queried using the regular query syntax.

```
SELECT Name, Edept FROM Emp WHERE ENo = 22217 AND EStart <= DATE '2011-01-02' AND EEnd > DATE '2011-01-02'
```

A simpler way to formulate the above query would be to employ one of the period predicates provided in SQL-2011 for expressing conditions involving periods:

`CONTAINS, OVERLAPS, EQUALS, PRECEDES,SUCCEEDS, IMMEDIATELY PRECEDES,` and `IMMEDIATELY SUCCEEDS.`

SQL 2011 Standard

The query could be expressed using the CONTAINS predicate:

```
SELECT Ename, Edept FROM Emp WHERE ENo =
22217 AND EPeriod CONTAINS DATE '2011-01-02'
```

```
SELECT Ename, Edept FROM Emp WHERE ENo =
22217 AND EStart < DATE '2011-01-01' AND
EEnd > DATE '2010-01-01'
```

(the above query uses the closed-open model)

The same query using the OVERLAPS predicate:

```
SELECT Ename, Edept FROM Emp WHERE ENo =
22217 AND EPeriod OVERLAPS PERIOD (DATE
'2010-01-01', DATE '2011-01-01')
```

SQL 2011 Standard

- System-versioned tables are used in applications that maintain a history of data changes either for business reasons, legal reasons, or both. A typical example of such applications is a banking application, where it is necessary to keep previous states of customer account information
- customers can be provided with a detailed history of their accounts.

Any update or delete of a row must automatically record the old state of the row before performing the update or delete.

SQL 2011 Standard

- The **system**, rather than the user, **maintains the start and end times of the periods** of the rows, and that users be unable to modify the content of historical rows or the periods associated with any of the rows.
- Any updates to the periods of rows in a system-versioned table must be performed only by the system as a result of updates to the non-period columns of the table or as a result of row deletions.

- guarantees that the recorded history of data changes cannot be altered, which is critical to meet auditing and compliance regulations.

SQL 2011 Standard

- Any table that contains a period definition with the standard-specified name, SYSTEM_TIME, and includes the keywords WITH SYSTEM VERSIONING in its definition is a system-versioned table.
- Users can pick any name they want for the names of columns that act as the start and end columns of the SYSTEM_TIME period. Though SQL-2011 allows the data types of the period start and end columns to be either DATE or a timestamp type (as long as the data types of both columns are the same), in practice, most implementations will provide the TIMESTAMP type with the highest fractional seconds precision as the data type for the system-time period start and end columns.

SQL 2011 Standard

```
CREATE TABLE Emp(
ENo INTEGER,
Sys_start TIMESTAMP(12) GENERATED ALWAYS AS
ROW START,
Sys_end TIMESTAMP(12) GENERATED ALWAYS AS
ROW END,
EName VARCHAR(30),
PERIOD FOR SYSTEM_TIME (Sys_start, Sys_end)
) WITH SYSTEM VERSIONING
```

Similar to application-time periods, system-time periods use closed-open period mode.

SQL 2011 Standard

2) INSERT into a system-versioned table automatically sets the value of Sys_start column to the transaction timestamp, a special value associated with every transaction and sets the value of Sys_end column to the highest value of the column's data type.

3) UPDATE and DELETE on system-versioned tables only operate on current system rows. Users are not allowed to:

- update or delete historical system rows.
- modify the system-time period start or the end time of both current system rows and historical system rows.

SQL 2011 Standard

At any given point in time, a row in a system-versioned table is regarded as current system row if the system-time period of that row contains the current time. A row that is not a current system row is regarded as a historical system row.

System-versioned tables differ from application-time period tables in the following respects:

1) In contrast to the application-time period tables,users are not allowed to assign or change the values of Sys_start or Sys_end columns; they are assigned (and changed) automatically by the database system. This is the reason why the definitions of Sys_start or Sys_end columns must include the keywords GENERATED ALWAYS.

SQL 2011 Standard

4) UPDATE and DELETE on system-versioned tables result in the automatic insertion of a historical system row for every current system row that is updated or deleted. An UPDATE statement on a system-versioned table first inserts a copy of the old row with its system-time period end time set to the transaction timestamp, indicating that the row ceased to be current as of the transaction timestamp. It then updates the row while changing its system-period start time to the transaction timestamp,indicating that the updated row to be the current system row as of the transaction timestamp

SQL 2011 Standard

A DELETE statement on a system-versioned table does not actually delete the qualifying rows; instead it changes the system-time period end time of those row to the transaction timestamp, indicating that those rows are no longer current as of the transaction timestamp.

SQL 2011 Standard

Primary key and referential constraints on system-versioned tables

The definition and enforcement of constraints on system-versioned tables is considerably simpler than the definition and enforcement of constraints on application-time period tables. This is because constraints on system-versioned tables need only be enforced on the current system rows. Historical system rows in a system-versioned table form immutable snapshots of the past.

SQL 2011 Standard

Any constraints that were in effect when a historical system row was created would have already been checked when that row was a current system row, so there is never any need to enforce constraints on historical system rows. Consequently, there is no need to include the system-period start and end columns or the period name in the definition of primary key and referential constraints on system-versioned tables. For example, the following

```
ALTER TABLE Emp ADD PRIMARY KEY (ENo)
```

SQL 2011 Standard

The above constraint ensures there exists exactly one current system row with a given ENo value. Similarly, the following ALTER TABLE statement specifies a referential constraint between Emp and Dept tables:

```
ALTER TABLE Emp ADD FOREIGN KEY
(Edept)REFERENCES Dept (DNo)
```

The above constraint is again enforced only on the current system rows of Emp and Dept tables.

SQL 2011 Standard

Because system-versioned tables are intended primarily for tracking historical data changes, queries on system-versioned tables often tend to be concerned with retrieving the table content as of a given point in time or between any two given points in time. SQL-2011 provides *three syntactic extensions* for this specific purpose. These are allowed only in queries on system-versioned tables.

The first extension is the **FOR SYSTEM_TIME AS OF** syntax that is useful for querying the table content as of a specified point in time.

SQL 2011 Standard

The other two extensions are
the period specified in the (FROM ... TO ...)
corresponds to a closed-open period model while the
period specified in the (BETWEEN ... AND ...) corresponds
to a closed-closed period model.

SQL 2011 Standard

Bitemporal tables

- combine the capabilities of both system-versioned and application-time period tables.

- the user is in charge of supplying values for the application-time period start and end columns.

- INSERT sets the value of system-time period start column to the transaction timestamp, and the value of system time period end column to the highest value of the column's data type.

- the UPDATE statement as well as UPDATE with FOR PORTION OF application-period can be used to modify the rows of bitemporal tables.

SQL 2011 Standard

- the DELETE statement as well as DELETE with FOR PORTION OF application-period can be used to delete rows from bitemporal tables.

- only current rows in system-time can be updated or deleted and a historical system row is automatically inserted for every current system row that is updated or deleted.

Queries on bitemporal tables can specify predicates on both application-time periods as well as system-time periods to qualify rows that will be returned as the query result.